

작성자: 박성현 (60222032 환경시스템공학)

1. 과제 개요 및 목표

- **프로젝트명:** 비트코인 리스크 매니저 V4.0 (최종판)
- **목표:** 기존에 개발한 3차 과제(V3.0) 코드에 14주차 핵심 문법인 2차원 리스트(2D List), 파일 입출력(txt), 예외 처리(try-except)를 융합하여, 프로그램이 외부 요인에도 튕기지 않고 데이터를 영구 보존할 수 있도록 견고하게 업그레이드한다.

2. AI(Gemini) 파트너십 및 단계별 문제 해결 과정

STEP 1. 새로운 문법 적용을 위한 뼈대 설계 논의

- **나의 질문 (Prompt):** "기존에 만든 3차 과제 비트코인 리스크 매니저 코드에, 이번에 배운 2차원 리스트 기능과 txt 파일 저장/불러오기 기능을 합치려고 해. 데이터가 계속 누적되려면 코드를 어떤 구조로 짜야 할까?"
- **AI의 조언:** 기존 1차원 변수를 이중 리스트 형태인 `trade_history_2d = []`로 변경하고, 거래가 끝날 때마다 `append([코인명, 레버리지, 변동률, 손익, 상태])` 형식으로 묶어서 넣는 방식을 제안받음.
- **적용 및 결과:** AI가 제시한 뼈대를 바탕으로, 3번 메뉴(리포트 출력)를 눌렀을 때 for문을 돌려 이중 리스트 안의 데이터를 표 형태로 깔끔하게 뽑아내고, 동시에 with open을 통해 `bitcoin_report.txt` 파일에 텍스트로 쓰기(Write) 모드가 작동하도록 코드를 직접 구성함.

STEP 2. 치명적 문법 에러(Syntax Error) 및 오타 디버깅

- **나의 질문 (Prompt):** "내가 구조를 잡고 코드를 직접 타이핑해서 실행해 봤는데, 문법 에러가 나면서 실행이 안 돼. 그리고 자꾸 변수 이름을 못 찾는다고 나오는데 뭐가 문제일까?"
- **AI의 조언 및 코드 리뷰:**
 1. `print(f'{record[1]x | ... }')` 부분에서 변수를 닫는 중괄호 `}`가 누락된 치명적인 문법 오류를 짚어줌.
 2. 전역 변수로 선언한 `account_balance`를 내가 `account_balnce`로 오타를 낸 부분을 찾아내어 스펠링을 통일하도록 안내해 줌.
 3. `while True:` 무한루프의 4번(종료) 메뉴에서 프로그램 탈출을 위한 `break`가 빠져있음을 지적해 줌.
- **적용 및 결과:** AI의 지적을 바탕으로 중괄호를 닫고, 오타를 모두 교정했으며, 저장할 때 글자가 붙어 나오는 문제를 해결하기 위해 `{record[3]:.2f} USDT, {record[4]}\n` 처럼 띄어쓰기와 콤마 포매팅을 스스로 추가하여 출력 화면을 완벽하게 다듬음.

STEP 3. 예외 처리(try-except)를 통한 튕김 현상 방어

- **나의 질문 (Prompt):** "코드를 다 고치고 실행했는데, 처음 시작할 때 불러올 txt 파일이 없으니까 `FileNotFoundError`가 뜨면서 튕겨. 그리고 입금액에 숫자가 아니라 문자를 실수로 쳤을 때 `ValueError`가 뜨면서 또 꺼지는데, 이거 어떻게 막아?"
- **AI의 조언:** 데이터가 입력되는 `input()` 구간과, 파일을 불러오는 `open()` 구간에 try-except 블록을 씌워 에러를 우회하는 방법을 설명해 줌.
- **적용 및 결과:** * 파일 읽기 함수(`load_data`)에 try-except `FileNotFoundError`를 적용하여, 파일이 없을 경우 에러 대신 "[안내] 새로 시작합니다"라는 문구가 뜨도록 수정함.
 - 자금 입금과 레버리지 입력 칸에 try-except `ValueError`를 씌워, 사용자가 실수로 한글이나 영어를 입력하면 프로그램이 꺼지는 대신 거래를 취소하고 메인 메뉴로 안전하게 돌아가도록

로직을 추가함.

3. 최종 완성 코드 (main.py)

```
Python
# 파일이름 :main.py
# 작 성 자 :박성현 60222032 환경시스템공학

account_balance = 0.0
trade_history_2d = []

def display_menu():
    print('\n' '=' 35)
    print('비트코인 리스크 매니저 V4.0(최종판)')
    print('=' 35)
    print('1.투자 자금 설정(입금)')
    print('2.선물 거래 시뮬레이션 시작')
    print('3.최종 자산 및 거래 내역 리포트')
    print('4.프로그램 종료')
    print('=' 35)

def load_data():
    global account_balance, trade_history_2d
    try
        with open 'bitcoin_report.txt' 'r' encoding='utf-8' as f:
            lines = f.readlines()
            if lines:
                print('\n[안내] 이전 저장된 데이터를 성공적으로 불러왔습니다.')
            except FileNotFoundError:
                print '\n[안내] 기존에 저장된 성적/거래 파일이 없습니다. 새로 시작합니다'

def start_trading():
    global account_balance, trade_history_2d

    if account_balance <= 0:
        print '잔고가 부족합니다. 먼저 1번 메뉴에서 자금을 입금해주세요.'
        return
```

```

print("\n[선물 거래 시뮬레이션]")
try
    coin_name = input '거래할 코인 이름(예: BTC,ETH)을 입력하세요: '
    leverage = int input '레버리지를 설정하세요(1~125): '
    change_rate = float input '예상 변동률(%)을 입력하세요: '
except ValueError
    print '[오류] 레버리지와 변동률은 반드시 숫자로 입력해야 합니다! 거래 취소!'
    return

profit_loss = (account_balance * (change_rate / 100) * leverage)

if change_rate > 0
    status = '수익실현'
elif change_rate < 0
    status = '손실발생'
    if abs(profit_loss) >= account_balance
        print "[위험] 투자 원금이 전액 소실되어 '강제 청산'되었습니다!"
        account_balance = 0
        trade_history_2d.append([coin_name, leverage, change_rate, profit_loss, '강제 청산'])
    return

else
    status = '횡보장(변동없음)'

account_balance += profit_loss
trade_history_2d.append([coin_name, leverage, change_rate, profit_loss, status])

print f'거래결과: {profit_loss:.2f} USDT ({status})'
print f'현재잔고: {account_balance:.2f} USDT'

def show_and_save_report():
    print("\n" + '📊' + ' 15')
    print('📊 [실시간 자산 및 거래 기록 리포트]')
    print(f'최종 보유 자산: {account_balance:.2f} USDT')
    print '-' 40
    print('코인명 | 레버리지 | 변동률 | 손익결과 | 상태')
    print '-' 40

for record in trade_history_2d:
    print f'{record[0]} | {record[1]}x | {record[2]}% | {record[3]:.2f} USDT, | {record[4]} '

print('📊' + ' 15')

```

```

try
    with open 'bitcoin_report.txt' 'w' encoding='utf-8' as f:
        f.write f'최종 자산: {account_balance:.2f} USDT\n'
        f.write '코인명,레버리지,변동률,손익,상태\n'
        for record in trade_history_2d:
            f.write f'{record[0]},{record[1]},{record[2]},{record[3]:.2f} USDT,{record[4]}\n'
        print "\n✅ 거래 내역이 'bitcoin_report.txt' 파일로 안전하게 저장되었습니다!"
except Exception as e:
    print f'파일 저장 중 오류가 발생했습니다: {e}'

load_data()

while True
    display_menu()
    choice = input '원하는 메뉴 번호를 선택하세요: '

    if choice == '1'
        try
            deposit_amount = float input '입금할 금액 (USDT)을 입력 하세요: '
            account_balance += deposit_amount
            print f'{account_balance:.2f} USDT 자산이 지갑에 충전 되었습니다'
        except ValueError:
            print '🚨 [오류] 입금액은 숫자로만 입력해주세요!'

    elif choice == '2'
        start_trading()

    elif choice == '3'
        show_and_save_report()

    elif choice == '4'
        print '\n프로그램을 안전하게 종료합니다. 이용해주셔서 감사합니다.'
        break

    else
        print '🚨 올바른지 않은 번호입니다. 1번부터 4번 사이의 메뉴를 선택해주세요!'

```